

Beste Teillösungen für ungelöste Aufgaben (DeepCoder)

Bemerkung zur Bachelorarbeit – Steve Leonel Yomi Mbiakop

3. August 2026

1 Ausgangslage

Wie in der separaten Bemerkung *“Warum liefern DeepCoder und DreamCoder für ungelöste Aufgaben kein Kandidatenprogramm?”* gezeigt, speicherte die DeepCoder-Suche (`deepcoder/search.py`) für eine Aufgabe nur dann ein Programm, wenn dieses **alle** gegebenen Beispiele exakt erfüllte. Scheiterte jeder während der Suche geprüfte Kandidat an mindestens einem Beispiel, blieb `solution = None` – und alle Strukturmetriken (`OperationScore`, `PositionScore`, `OrderScore`, `EditScore`, `PartialCorrectness`) waren für diese Aufgabe rechnerisch immer 0, unabhängig davon, wie nah die Suche tatsächlich gekommen war.

Dieses Dokument beschreibt, wie DeepCoder so erweitert wurde, dass der **beste gefundene Teiltreffer** zusätzlich gespeichert und für die Metrikberechnung genutzt wird – ausschließlich für DeepCoder (nicht für DreamCoder, siehe Abschnitt 5).

2 Lösung

2.1 `deepcoder/search.py`: Teiltreffer während der Suche verfolgen

Neue Hilfsfunktion, die zählt, wie viele Beispiele ein Kandidat erfüllt (statt wie `is_solution` beim ersten Fehlschlag abubrechen):

```
def count_matches(program, examples):
    return sum(1 for inputs, output in examples
               if program(*inputs) == output)
```

In `dfs()` wird für jeden Kandidaten, der *nicht* alle Beispiele erfüllt, dessen Trefferzahl bestimmt. Übertrifft sie den bisher besten Wert, wird der Kandidat gemerkt:

```
ns = { 'nb_steps': 0,
        'solution': None,
        'gas' : gas,
        'best_partial': None,
        'best_partial_count': 0 }
...
try:
    if is_solution(p_base, examples):
        ns['solution'] = p_base
        return True
    match_count = count_matches(p_base, examples)
    if match_count > ns['best_partial_count']:
        ns['best_partial'] = p_base
        ns['best_partial_count'] = match_count
except (NullInputError, OutputOutOfRangeException):
```

```

    return
    ...
return ns['solution'], ns['nb_steps'], ns['best_partial'], ns['best_partial_count']

```

Findet sich nie ein Kandidat, der auch nur ein Beispiel trifft, bleibt `best_partial` bei `None` – die Aufgabe fällt dann exakt auf das alte Verhalten zurück (siehe Abschnitt 3). Der alternative Suchmodus `sort_and_add()` wurde analog erweitert und vergleicht den besten Teiltreffer über alle geprüften Kontexte hinweg.

2.2 scripts/solve-problems.py: neue Ausgabespalten

Für jede Aufgabe werden jetzt zwei zusätzliche Spalten geschrieben, befüllt nur wenn die Aufgabe *nicht* gelöst wurde:

- `best_partial_solution` – Programmstring des besten Teiltreffers (oder leer).
- `best_partial_match_fraction` – Anteil erfüllter Beispiele (z. B. 0.4 bei 2 von 5 Beispielen).

2.3 scripts/export_results_to_csv.py: Spalten durchreichen

Reicht die beiden neuen Spalten unverändert von der `.h5`- in die `.csv`-Datei durch (rückwärtskompatibel: ältere `.h5`-Dateien ohne diese Spalten funktionieren weiterhin).

2.4 scripts/compute_metrics.py: Best-Effort-Metriken

Neue Funktion, die entscheidet, welches Programm für den strukturellen Vergleich herangezogen wird:

```

def best_effort_program(row, has_partial_column):
    if row["solved"]:
        return row["solution"]
    if has_partial_column and is_present(row.get("best_partial_solution")):
        return row["best_partial_solution"]
    return None

```

Auf Basis dieses Programms werden fünf neue Spalten berechnet – `best_effort_operation_score`, `_position_score`, `_order_score`, `_edit_score` und `_partial_correctness` – mit denselben Metrik-Funktionen, die bereits für gelöste Aufgaben verwendet werden. Die bisherigen Spalten (`accuracy`, `partial_correctness` usw.) bleiben unverändert erhalten; die neuen Spalten kommen zusätzlich hinzu.

2.5 Notebook: zwei neue Diagramme

`deepcoder_metric_analysis.ipynb` zeigt jetzt zusätzlich zum bestehenden Gesamtvergleich zwei eigene Balkendiagramme:

- **Nur gelöste Aufgaben** – Strukturmetriken, ohne vs. mit Training.
- **Nur ungelöste Aufgaben** – dieselben Metriken, aber auf Basis von `best_effort_*`, statt pauschal 0.

3 Sonderfall: gar kein Kandidat gefunden

Manche Aufgaben bleiben auch mit dieser Erweiterung bei 0: wenn die Suche innerhalb des Budgets (`gas`) *nie* ein Programm gefunden hat, das auch nur ein einziges Beispiel erfüllt, bleibt `best_partial_solution` leer, `best_effort_program` liefert `None`, und alle `best_effort_*`-Metriken werden korrekt zu 0 (weil `tokenize_program(None)` eine leere Token-Liste ergibt –

dieselbe Rechnung wie zuvor bei einer leeren `solution`). Kein Fehler, kein Absturz, keine künstliche Verzerrung – nur eine ehrliche Rückkehr zum alten Zustand für genau diese Aufgaben.

4 Validierung

Testlauf auf `T=2_test` (100 Aufgaben, `gas = 1000`, ohne Training):

	vorher	nachher
Gelöste Aufgaben	53/100	53/100 (unverändert)
Ungelöste Aufgaben mit Teiltreffer	–	25/47
Ungelöste Aufgaben ganz ohne Kandidat	–	22/47
$\emptyset$ <code>partial_correctness</code> (ungelöst)	0,0000	–
$\emptyset$ <code>best_effort_partial_correctness</code> (ungelöst)	–	0,2556

Die Lösungsrate (`solved`) blieb exakt gleich – die Erweiterung verändert nicht, *ob* eine Aufgabe als gelöst gilt, sondern liefert für die verbleibenden Fälle zusätzliche, ehrliche Information über die strukturelle Nähe der Suche zum Referenzprogramm.

Konkretes Beispiel (Aufgabe `LIST|COUNT,<0,0|TAKE,1,0`, ungelöst): der beste gefundene Teiltreffer erfüllt 2 von 5 Beispielen (40 %) und erhält dadurch einen echten `best_effort_*`-Score statt einer pauschalen Null.

5 Umfang: nur DeepCoder

Diese Erweiterung wurde bewusst nur für DeepCoder umgesetzt. Dieser Abschnitt begründet das nicht nur, sondern zeigt auch konkret, was für DreamCoder nötig wäre, und warum die Entscheidung dagegen eine bewusste Kosten-Nutzen-Abwägung war – keine Verlegenheitslösung.

5.1 Warum DreamCoder strukturell anders liegt

Bei DeepCoder reichte eine reine Python-Änderung, weil die komplette Suche in Python läuft (`deepcoder/search.py`). Bei DreamCoder liegt die eigentliche Enumeration dagegen im kompilierten OCaml-Solver. Wie in der separaten Bemerkung gezeigt, entscheidet dort `solvers/task.ml` pro Kandidat binär:

```
let logLikelihood = tasks.(j).log_likelihood p in
if is_valid logLikelihood then begin
  Heap.add hits.(j) {hit_program = ...; hit_likelihood = logLikelihood; ...}
end
```

Ein Kandidat mit `logLikelihood = neg_infinity` (scheitert an mindestens einem Beispiel) besteht `is_valid` nicht und wird *nie* in die Heap aufgenommen, die später als JSON an Python zurückgegeben wird (`solvers/solver.ml`, `export_frontiers`). Der Python-Wrapper (`solveForTask_ocaml` in `dreamcoder/enumeration.py`) bekommt also gar keine Kandidaten mit Teiltreffern zu Gesicht – sie existieren zu keinem Zeitpunkt außerhalb des OCaml-Prozesses.

5.2 Was die Umsetzung technisch erfordern würde

Um das DeepCoder-Prinzip (bester Teiltreffer statt Verwerfen) auch für DreamCoder umzusetzen, wären mindestens vier zusammenhängende Änderungen nötig:

1. `solvers/task.ml`, `supervised_task`: Eine zweite, graduelle Bewertungsfunktion ergänzen, die zählt, wie viele Beispiele `p` erfüllt (analog zu `count_matches` in DeepCoder), statt beim ersten Fehlschlag mit `log 0.0` abubrechen.

2. `solvers/task.ml`, `enumerate_for_tasks`: Pro Aufgabe einen zweiten, von `hits` getrennten Merker für den bisher besten Teiltreffer einführen (die `is_valid`-Prüfung darf für `hits` unverändert bleiben, damit sich an gelösten Aufgaben nichts ändert).
3. `solvers/solver.ml`, `export_frontiers`: Das JSON-Antwortformat um ein optionales Feld für den besten Teiltreffer je Aufgabe erweitern.
4. `dreamcoder/enumeration.py`, `solveForTask_ocaml`: Das neue Feld einlesen und (ähnlich wie bei DeepCoder) getrennt von der regulären `Frontier` als `best_partial`-Information an die Auswertungs-Pipeline weiterreichen.

Jede dieser vier Änderungen ist für sich genommen nicht kompliziert – das Muster ist ja bei DeepCoder bereits erfolgreich erprobt. Der eigentliche Aufwand liegt nicht in der Logik, sondern in der **Bauumgebung**, siehe unten.

5.3 Konkret geprüfter Befund: keine Build-Umgebung vorhanden

```
$ which ocaml ocamlfind dune opam
(nichts gefunden)
$ opam --version
bash: opam: command not found
$ find / -iname '*opam*'
/etc/apparmor.d/opam (AppArmor-Profil, kein Programm)
/usr/share/vim/.../opam.vim (Vim-Syntaxdatei, kein Programm)
```

In der WSL-Umgebung ist aktuell *kein* OCaml-Build-Toolchain installiert. Die vorhandenen Binaries `solver` und `compression` sind vorgefertigt (Änderungsdatum 10. Juli, aus der ursprünglichen Repository-Einrichtung) und lokal nicht neu baubar. Das Build-System selbst (`solvers/Makefile`) verrät zusätzlich, mit welchem – inzwischen unüblichen – Werkzeug gebaut wurde:

```
c = corebuild -pkg yojson -pkg re2 -tag unsafe -tag "optimize(3)"
```

`corebuild` ist ein älteres, auf Jane Streets `Core`-Bibliothek zugeschnittenes Build-Werkzeug (Vorgänger des heute üblichen `dune`). Zusätzliche Abhängigkeit: `re2`, eine OCaml-Anbindung an Googles gleichnamige C++-Regex-Bibliothek – solche nativen Bindings sind erfahrungsgemäß besonders versionsempfindlich beim Neuaufsetzen. Es existiert weder ein Dockerfile noch ein Setup-Skript im Repository, das den ursprünglichen Build-Prozess dokumentiert oder reproduzierbar macht.

5.4 Kosten-Nutzen-Abwägung

Vor der eigentlichen Code-Änderung (Abschnitt 5.2) stünde also erst der Aufbau einer kompletten OCaml-Toolchain: `opam` installieren, einen zur `Core`-Version passenden OCaml-Switch anlegen, `core`, `yojson` und `re2` samt alle transitiven Abhängigkeiten installieren, und danach verifizieren, dass sich der *unveränderte* Code überhaupt noch baut, bevor die eigentliche Änderung überhaupt begonnen werden kann. Realistischer Aufwand: 1 + Stunden, mit ungewissem Ausgang – gerade `re2` ist ein bekannter Quell für Build-Fehler bei neu aufgesetzten OCaml-Umgebungen.

Angesichts dieses Aufwands-Nutzen-Verhältnisses – und angesichts der Erfahrung aus der heutigen Arbeit, wie instabil die WSL2-Umgebung bereits bei reinen Python-Läufen war (Speicherabstürze, siehe separate Bemerkung) – wurde bewusst entschieden, diese Erweiterung **nicht** umzusetzen. Die vier oben skizzierten Schritte bleiben als konkret ausformulierter, sofort umsetzbarer nächster Schritt festgehalten, falls zu einem späteren Zeitpunkt eine funktionierende OCaml-Umgebung zur Verfügung steht.

Wichtig für die Einordnung der Ergebnisse: Diese Entscheidung schränkt *nicht* die Gültigkeit der bisherigen DreamCoder-Ergebnisse ein. Sie bedeutet lediglich, dass für DreamCoders

ungelöste Aufgaben – anders als bei DeepCoder – weiterhin nur der binäre Zustand “gelöst/nicht gelöst” und keine graduelle Teilkorrektheit verfügbar ist.